

Precomputed Path-Traced Light Transport for Low-Cost Real-Time Global and Volumetric Illumination

xt9y

Abstract—High-quality global illumination and volumetric light transport are commonly associated with substantial real-time sampling cost, large three-dimensional lighting structures, or specialized ray-tracing hardware. This paper presents a different design point intended for severely constrained graphics hardware: path tracing is used primarily as an offline transport compiler, and runtime shading consumes compact precomputed representations. The method decomposes lighting by spatial and angular frequency. Static diffuse transport is stored in path-traced lightmaps; low-frequency directional radiance is stored in second-order spherical-harmonic probe fields; direct sunlight in participating media is represented by an offline ray-traced sun-visibility field compressed into beam cells; and indirect volumetric radiance is reconstructed from sparse path-traced radiance probes. Homogeneous-medium segments are integrated analytically, avoiding dense volumetric ray marching. A single stackless BVH traversal implementation is shared by all bake stages. The resulting runtime contains no continuous global-illumination path tracing and no per-sample volumetric shadow rays. We describe the transport formulation, compressed representations, GPU bake kernels, runtime reconstruction, memory model, and limitations of this architecture, with specific reference to a compact C/SDL_GPU renderer.

I. INTRODUCTION

Modern real-time renderers often solve lighting by repeatedly evaluating visibility and transport during every frame. This approach is flexible, but it couples image quality to runtime ray count, volumetric sample density, shadow-map resolution, or the availability of dedicated ray-intersection hardware. That coupling is unattractive for renderers that must execute on weak integrated GPUs or older discrete hardware.

The central observation of this work is that a large fraction of lighting transport in a static environment is invariant with respect to the camera. Surface interreflection, spatially varying indirect radiance, and direct-sun visibility can therefore be evaluated once and converted into representations whose runtime cost is largely independent of the original number of traced paths. A four-bounce surface path and a twenty-bounce surface path may both become a single texel. Thousands of directional probe paths may become nine RGB coefficients. Millions of high-resolution sun-visibility rays may become a few thousand oriented beam cells.

The proposed system is designed around four transport layers:

- 1) path-traced lightmaps for static diffuse global illumination;

- 2) path-traced spherical-harmonic (SH) radiance probes for moving-object indirect lighting;
- 3) an offline sun-visibility bake compressed into beam cells for direct volumetric shafts; and
- 4) path-traced SH radiance probes for indirect volumetric illumination.

The decomposition is deliberate. Low-order SH is efficient for smooth directional functions but unsuitable for narrow visibility discontinuities. Direct sunlight passing through a small aperture is therefore kept out of the SH representation and encoded geometrically. Conversely, smooth indirect illumination is not stored at surface-like spatial resolution.

This architecture is implemented conceptually around Dustmite, a small C renderer using SDL_GPU, a static stackless BVH, and a GPU path-traced lightmap baker. The method does not require hardware ray tracing. Ray traversal is expensive only during preprocessing; the target runtime machine can load the baked products directly.

The contributions of the design are:

- a unified bake-time transport formulation shared by lightmaps and radiance probes;
- a shared SH radiance representation usable by both moving surfaces and participating media;
- a sun-space visibility decomposition that converts offline ray queries into compact beam cells;
- analytic evaluation of homogeneous volumetric segments, removing conventional fine-step ray marching; and
- a runtime cost structure dominated by texture fetches, interpolation, low-order SH evaluation, and rasterization.

No empirical performance claim is made in this paper. The performance discussion is analytical and defines the measurements required for a subsequent implementation evaluation.

II. RELATED WORK

A. Precomputed Light Transport

Precomputed radiance transfer (PRT) demonstrated that expensive global transport can be evaluated in a preprocessing stage and projected into a compact basis for efficient runtime reconstruction [2]. Low-order spherical harmonics are particularly effective for diffuse, low-frequency directional illumination because the shading

integral can be reduced to small coefficient operations. Sloan later summarized practical evaluation, product, and ringing considerations for real-time SH lighting [3].

The present work follows the same broad principle—precompute transport rather than rediscover it every frame—but uses different representations for different frequency regimes. Static surface transport is stored directly in UV space, smooth directional transport is stored in SH, and sharp direct-sun volumetric visibility is stored as explicit beam geometry.

B. Volumetric Lighting

Single-scattering volumetric rendering integrates incident radiance along camera rays while accounting for medium extinction and phase response. Engelhardt and Dachsbacher exploited the observation that volumetric visibility is often smooth except near shadow discontinuities, concentrating sampling around important regions [4]. Hillaire presented a unified physically based volumetric framework for real-time production rendering and emphasized consistent participating-media lighting and temporal/spatial reuse [5].

Our method makes a stronger static-scene assumption. Instead of discovering direct-sun visibility discontinuities at runtime, it identifies them during preprocessing and stores the illuminated domains explicitly. This exchanges dynamic flexibility for low runtime cost.

C. Path Tracing and GPU Parallelism

The rendering equation introduced a unified formulation of light transport [1]; modern path tracers estimate this equation using Monte Carlo methods. GPU implementations exploit the independence of large numbers of rays and samples [8]. In the proposed architecture, the same parallelism is used during content preprocessing rather than for every displayed frame.

D. Participating Media

The notation and transport equations in this paper follow standard physically based volume rendering formulations [7]. The Henyey–Greenstein phase function [6] is used as a compact single-parameter model for anisotropic scattering.

III. SYSTEM OVERVIEW

Figure 1 shows the complete architecture. The static scene is converted to a BVH once. A shared GPU transport kernel produces three distinct classes of baked data: UV-space lightmaps, SH radiance probes, and sun-space visibility. Runtime shading does not traverse the static BVH for these lighting effects.

A. Static and Dynamic Assumptions

The design assumes that the principal environment geometry, sun direction, sky distribution, and participating-media regions remain fixed for a given bake. Moving objects may receive indirect illumination from probes, but their motion does not modify precomputed static transport.

Direct dynamic-object shadowing is orthogonal to the method and can be supplied by a small dynamic shadow map if required.

This restriction is intentional. The target is not a universally dynamic lighting system; it is a renderer that converts static scene information into unusually high visual quality at low frame cost.

B. Persistent Bake Asset

Bake-time and runtime execution should be separated physically. A target device should not need to execute the path tracer during startup. The scene compiler therefore produces a persistent baked-lighting asset containing the lightmap, radiance-probe fields, medium descriptors, sun basis, and compressed beam cells.

A minimal file header is:

```
typedef struct dm_bake_header {
    uint32_t magic;
    uint32_t version;

    uint64_t scene_hash;
    uint64_t config_hash;

    uint32_t lightmap_width;
    uint32_t lightmap_height;
    uint32_t object_probe_count;
    uint32_t volume_probe_count;
    uint32_t sun_beam_count;

    uint64_t lightmap_offset;
    uint64_t object_probe_offset;
    uint64_t volume_probe_offset;
    uint64_t sun_beam_offset;
} dm_bake_header;
```

The scene hash prevents stale lighting from being loaded after geometry changes. The configuration hash includes transport parameters such as the sun, sky, bounce count, probe placement, medium coefficients, and representation versions.

IV. UNIFIED BAKE-TIME TRANSPORT

A. Radiance and Diffuse Irradiance

Incoming radiance at position $\mathbf{x}$ from direction ω is written as

$$L_i(\mathbf{x}, \omega). \quad (1)$$

Diffuse irradiance for a surface normal $\mathbf{n}$ is

$$E(\mathbf{x}, \mathbf{n}) = \int_{\Omega^+} L_i(\mathbf{x}, \omega) \max(0, \mathbf{n} \cdot \omega) d\omega. \quad (2)$$

For a Lambertian material with diffuse albedo ρ ,

$$L_o = \frac{\rho}{\pi} E. \quad (3)$$

The same incoming-radiance function is useful beyond surfaces. A volumetric phase function replaces the cosine kernel in (2), which motivates a common directional representation for surface and volumetric probes.

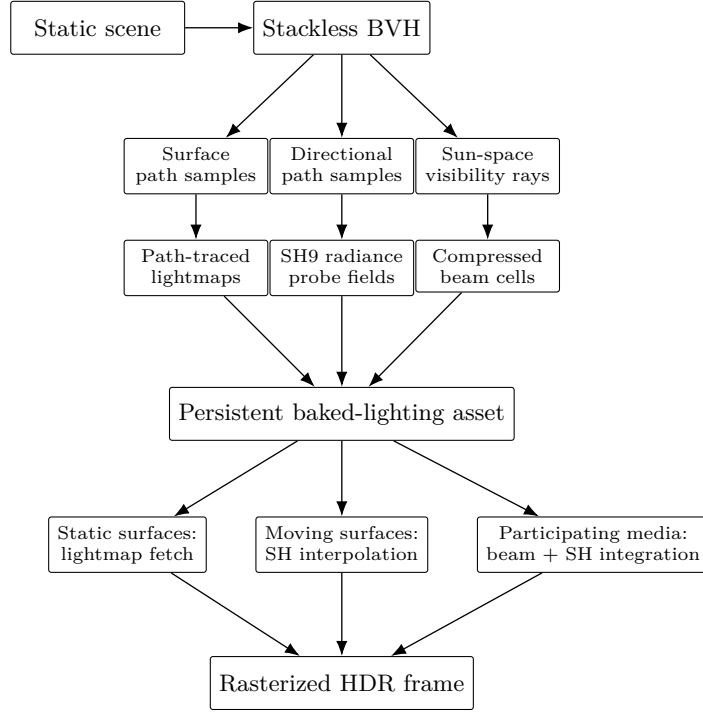


Fig. 1. Overview of the proposed transport pipeline. Expensive visibility and multi-bounce transport are resolved offline. The runtime consumes representations matched to the frequency and spatial structure of each lighting component.

B. Monte Carlo Estimation

For samples x_i drawn from probability density $p(x)$,

$$\int f(x) dx \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}. \quad (4)$$

Diffuse path bounces use cosine-weighted hemisphere sampling,

$$p(\omega) = \frac{\cos \theta}{\pi}, \quad (5)$$

which cancels the Lambertian BRDF's cosine/PDF factors and yields a compact throughput update.

C. Shared Path Kernel

Lightmaps and SH probes should not maintain separate transport implementations. Both require a function that estimates radiance arriving along a ray. A minimal GPU form is:

```

float3 trace_radiance(
    float3 origin,
    float3 direction,
    inout uint rng,
    uint max_bounces)
{
    float3 L = 0.0f;
    float3 beta = 1.0f;

    for (uint bounce = 0; bounce < max_bounces; ++bounce)
    {
        Hit hit;
        if (!trace_scene(origin, direction, hit))
        {
            L += beta * sky_radiance(direction);
            break;
        }

        Material m = material_at(hit);
        L += beta * m.emissive;

        /* Sun is sampled explicitly after a surface hit. */
        L += beta * sample_direct_sun(hit, rng);

        direction = cosine_hemisphere(hit.normal, rng);
        beta *= m.diffuse_albedo;

        origin = hit.position + hit.normal * TRACE_EPSILON;

        if (bounce >= 2u)
        {
            float q = max(beta.r, max(beta.g, beta.b));
            q = clamp(q, 0.05f, 0.95f);

            if (random01(rng) > q)
                break;

            beta /= q;
        }
    }

    return L;
}

```

The sun disk is excluded from the environment term. Direct sun is sampled explicitly at surface vertices of a path. As a result, a probe’s primary ray escaping directly to the sky does not duplicate the direct volumetric sun term encoded by beam cells, while sunlight that reflects from a surface is naturally included in indirect radiance.

D. Stackless BVH Traversal

Dustmite’s bake-time traversal uses threaded BVH nodes containing a child and a “next” link. The resulting early-out shadow test requires no dynamic traversal stack.

```
bool occluded(float3 ro, float3 rd, float max_t)
{
    uint node = 0u;

    while (node != INVALID_NODE)
    {
        BvhNode n = Nodes[node];

        if (!ray_box(ro, rd, n, max_t))
        {
            node = n.meta.y; /* next */
            continue;
        }

        if (n.meta.w != 0u)
        {
            for (uint i = 0; i < n.meta.w; ++i)
            {
                float t;
                if (ray_triangle(
                    ro, rd,
                    Triangles[n.meta.z + i],
                    max_t, t))
                    return true;
            }

            node = n.meta.y;
            continue;
        }

        node = n.meta.x; /* descend */
    }

    return false;
}
```

The same BVH buffers are consumed by lightmap, probe, and sun-visibility bake kernels. Runtime lighting does not require them.

V. STATIC DIFFUSE TRANSPORT

Static diffuse geometry is the highest-spatial-frequency indirect-lighting receiver in the system. It is therefore stored in a surface parameterization rather than in a sparse world-space field.

For every valid lightmap texel, the baker reconstructs world-space position and normal, traces one or more paths, and progressively averages the radiance estimate. If $\bar{L}_n$ is the mean after n samples,

$$\bar{L}_{n+1} = \frac{n\bar{L}_n + L_{n+1}}{n+1}. \quad (6)$$

The final atlas is filtered conservatively and dilated across UV-island borders. The runtime cost is one lightmap sample in the normal material-shading path. Crucially, that cost is independent of the number of path samples or bounce depth used during baking.

A. Bake Material Fidelity

A geometry-only BVH is sufficient for visibility but not for colored multi-bounce transport. Each bake-visible triangle must identify at least a material index. A minimal representation is

```
typedef struct bake_triangle {
    float a[4], b[4], c[4];
    float normal[4];

    uint32_t material;
    uint32_t _pad[3];
} bake_triangle;
```

A first implementation may use constant base-color and emissive factors. Texture-accurate bounces can later be supported using bake-time texture atlases without changing the runtime representation.

VI. RADIANCE PROBE FIELDS

A. Spherical-Harmonic Representation

A cubemap is unnecessarily expensive for the smooth directional radiance targeted by the probe fields. We instead project incoming radiance into second-order real spherical harmonics (SH9). The nine basis functions used in the implementation are

$$Y_0 = 0.282095, \quad (7)$$

$$Y_1 = 0.488603y, \quad Y_2 = 0.488603z, \quad Y_3 = 0.488603x, \quad (8)$$

$$Y_4 = 1.092548xy, \quad Y_5 = 1.092548yz, \quad (9)$$

$$Y_6 = 0.315392(3z^2 - 1), \quad (10)$$

$$Y_7 = 1.092548xz, \quad Y_8 = 0.546274(x^2 - y^2). \quad (11)$$

For uniformly distributed sphere samples, coefficient c_j is estimated as

$$c_j \approx \frac{4\pi}{N} \sum_{i=1}^N L_i(\omega_i) Y_j(\omega_i). \quad (12)$$

The exact basis code used by both bake and runtime is:

```
void sh9_basis(float3 d, out float sh[9])
{
    d = normalize(d);
    float x = d.x, y = d.y, z = d.z;

    sh[0] = 0.2820947918f;
    sh[1] = 0.4886025119f * y;
    sh[2] = 0.4886025119f * z;
    sh[3] = 0.4886025119f * x;

    sh[4] = 1.0925484306f * x * y;
    sh[5] = 1.0925484306f * y * z;
    sh[6] = 0.3153915653f * (3.0f*z*z - 1.0f);
    sh[7] = 1.0925484306f * x * z;
    sh[8] = 0.5462742153f * (x*x - y*y);
}
```

B. Shared Probe Representation

Object GI and indirect volumetric GI store the same physical quantity: directional incoming radiance. They can therefore share the same coefficient representation:

```
typedef struct dm_sh9_rgb {
    float c[9][3];
} dm_sh9_rgb;

typedef struct dm_radiance_probe {
    float position[3];
    float validity;
    dm_sh9_rgb radiance;
} dm_radiance_probe;
```

The two fields may use different spatial spacing, but they do not need different path-tracing kernels or file formats.

At FP16 precision, SH9 RGB requires

$$9 \times 3 \times 2 = 54 \text{ bytes} \quad (13)$$

per probe before metadata. This makes several thousand probes inexpensive compared with a conventional lightmap.

C. Probe Baking

Each valid probe independently samples directions over the sphere, evaluates the shared transport kernel, projects the returned radiance according to (12), and reduces the coefficients. One workgroup per probe maps naturally onto GPU execution: workgroup lanes trace independent rays and cooperatively reduce nine RGB coefficient sums.

A regular grid is used initially because it eliminates runtime spatial search. Given grid origin $\mathbf{o}$ and spacing s ,

$$\mathbf{q} = \frac{\mathbf{x} - \mathbf{o}}{s}. \quad (14)$$

The eight surrounding probes are blended using standard trilinear weights. Invalid probes near solid geometry are skipped and the surviving weights are renormalized.

D. Dynamic Diffuse Surfaces

To recover diffuse irradiance, raw radiance SH is convolved with the clamped-cosine kernel. For SH bands $l = 0, 1, 2$, the factors are

$$A_0 = \pi, \quad A_1 = \frac{2\pi}{3}, \quad A_2 = \frac{\pi}{4}. \quad (15)$$

For extremely weak hardware, the eight-probe interpolation can be performed once per moving object rather than per fragment. The resulting nine RGB coefficients are uploaded with the object constants. Diffuse irradiance can then be evaluated per vertex and interpolated across the triangle.

This is intentionally different from a general dynamic GI system. The representation is designed to make moving objects belong visually to a path-traced static environment at negligible transport cost.

VII. PARTICIPATING-MEDIA MODEL

For a homogeneous medium,

$$\sigma_t = \sigma_a + \sigma_s, \quad (16)$$

where σ_a is absorption, σ_s is scattering, and σ_t is extinction. Beer-Lambert transmittance over distance d is

$$T_r(d) = e^{-\sigma_t d}. \quad (17)$$

The direct and indirect volumetric systems both exploit piecewise-homogeneous media because complete ray segments can then be integrated analytically.

A. Phase Function

The Henyey-Greenstein phase function is

$$p(\mu) = \frac{1 - g^2}{4\pi(1 + g^2 - 2g\mu)^{3/2}}, \quad (18)$$

where $\mu = \cos \theta$. Positive g produces forward scattering; $g = 0$ is isotropic.

B. Analytic Segment Integration

Consider a camera ray crossing a homogeneous illuminated segment over distances $[a, b]$. If incident radiance and phase value are constant over the segment, the in-scattered contribution is

$$L_{\text{seg}} = L_i p \sigma_s \int_a^b e^{-\sigma_t t} dt \quad (19)$$

$$= L_i p \frac{\sigma_s}{\sigma_t} (e^{-\sigma_t a} - e^{-\sigma_t b}). \quad (20)$$

Equation (20) replaces a conventional loop of many fixed ray-march samples. Its stable zero-extinction limit is

$$L_{\text{seg}} \approx L_i p \sigma_s (b - a). \quad (21)$$

The corresponding implementation is compact:

```
float3 integrate_segment(
    float3 incident,
    float phase,
    float sigma_s,
    float sigma_t,
    float a,
    float b)
{
    if (b <= a)
        return 0.0f;

    if (sigma_t < 1.0e-5f)
        return incident * phase * sigma_s * (b - a);

    float Ta = exp(-sigma_t * a);
    float Tb = exp(-sigma_t * b);

    return incident *
        (phase * sigma_s / sigma_t) *
        (Ta - Tb);
}
```

VIII. PRECOMPUTED DIRECT-SUN VISIBILITY

A. Sun-Space Parameterization

For a directional sun, all visibility rays are parallel. Let $\mathbf{W}$ be the normalized sun-ray direction and let $\mathbf{U}$ and $\mathbf{V}$ complete an orthonormal basis. A world point $\mathbf{x}$ is transformed to sun coordinates

$$(u, v, w) = (\mathbf{U} \cdot \mathbf{x}, \mathbf{V} \cdot \mathbf{x}, \mathbf{W} \cdot \mathbf{x}). \quad (22)$$

For each (u, v) sample, the baker traces along $\mathbf{W}$ and records the directly illuminated interval inside a participating-medium region. If the medium occupies

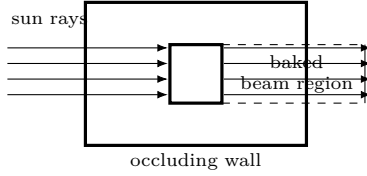


Fig. 2. Direct volumetric sunlight is represented only where offline sun-space visibility permits it. Apertures arise as depth discontinuities; the surviving interval field is compressed into oriented beam cells.

$[w_0, w_m]$ along that ray and the first opaque hit is w_h , the directly lit interval is

$$[w_0, \min(w_m, w_h - \epsilon)]. \quad (23)$$

An empty interval produces no beam data.

B. Apertures Emerge from Visibility

The system never needs a semantic “window” primitive. A wall produces nearby sun columns whose first-hit depths are shallow. A window produces columns whose first hit is much farther away. The visibility discontinuity is therefore present directly in the traced depth/interval field.

This property is essential for small apertures: geometric detail is detected according to the physical resolution of the offline sun sampling, not the current screen resolution.

C. Finite Sampling and Minimum Feature Size

A finite sample field cannot detect an arbitrarily small aperture. The baker therefore exposes a physical minimum sun feature size $s_{\min}$. If a projected sun-space extent of width W is represented by N samples,

$$N \geq \frac{W}{s_{\min}}. \quad (24)$$

Unlike a real-time shadow map, the resulting high-resolution field need not be retained. It can be processed in tiles, immediately compressed, and discarded.

D. Beam Cells

Neighboring lit sun columns are merged into axis-aligned boxes in sun space:

```
typedef struct dm_sun_beam_cell {
    float u0, u1;
    float v0, v1;
    float w0, w1;

    uint32_t medium_index;
    uint32_t _pad0;
} dm_sun_beam_cell;
```

The world-space interpretation is an oriented box aligned with the sun direction. Rectangular regions are recursively merged while the variation in entry and exit depths remains below a prescribed tolerance. Regions containing a visibility discontinuity remain subdivided.

IX. RUNTIME DIRECT VOLUMETRICS

All beam cells can be drawn as instanced box proxies into a low-resolution additive volumetric target. For each covered fragment, the shader reconstructs the camera ray, transforms it into sun space, intersects it with the corresponding box, clamps the interval to the opaque scene depth, evaluates the HG phase function, and applies (20).

A conceptual shader is:

```
float3 direct_beam(
    float3 camera_world,
    float3 view_dir,
    float scene_depth,
    BeamCell cell,
    Medium medium)
{
    float3 ro = world_to_sun(camera_world);
    float3 rd = world_dir_to_sun(view_dir);

    float t0, t1;
    if (!ray_box_interval(
        ro, rd,
        cell.bmin, cell.bmax,
        t0, t1))
        return 0.0f;

    t0 = max(t0, 0.0f);
    t1 = min(t1, scene_depth);

    if (t1 <= t0)
        return 0.0f;

    float mu = dot(-view_dir, -sun_direction);
    float phase = phase_hg(mu, medium.g);

    return integrate_segment(
        sun_color * sun_intensity,
        phase,
        medium.sigma_s,
        medium.sigma_s + medium.sigma_a,
        t0, t1);
}
```

Because each contribution includes transmittance from the camera to the absolute interval distances, disjoint direct-scattering beam segments are additive. The beam proxies do not require alpha sorting as long as they do not encode overlapping copies of the same illuminated region.

X. INDIRECT VOLUMETRIC RADIANCE

Direct sunlight is high frequency and retained explicitly. Bounced radiance is smoother and is reconstructed from the sparse volume-probe field.

The radiance SH coefficients are convolved with the phase function rather than the Lambertian cosine kernel. For a rotationally symmetric Henyey–Greenstein lobe, the SH band response is proportional to g^l . At second order,

$$k_0 = 1, \quad k_1 = g, \quad k_2 = g^2. \quad (25)$$

The view-direction SH basis is evaluated, bands are scaled according to (25), and nearby probe coefficients are interpolated spatially. The resulting directional radiance is then integrated over the camera-ray segment using (20).

Two runtime realizations are compatible with the representation. A coarse grid DDA can traverse one large segment per probe cell, or the grid cells can be drawn as box proxies in the same style as direct beam cells. In both

cases, the integration step length is the coarse probe-cell size rather than a fine fixed ray-march increment.

XI. RUNTIME COMPOSITION

For an opaque surface at camera distance d_s , homogeneous-medium transmittance is

$$T_s = e^{-\sigma_t d_s}. \quad (26)$$

The final HDR color is

$$C = C_{\text{surface}} T_s + L_{\text{direct volume}} + L_{\text{indirect volume}}. \quad (27)$$

The normal frame therefore remains predominantly rasterized:

```
opaque raster
  static -> lightmap
  moving -> interpolated SH GI

direct beam-cell raster
indirect volume-cell evaluation
depth-aware volumetric upsample
HDR composition
post processing
```

There is no static-scene path tracing in this sequence.

XII. IMPLEMENTATION IN A COMPACT SDL_GPU RENDERER

A. GPU Resource Model

A minimal runtime resource set contains

```
Texture2D          Lightmap;
StructuredBuffer<Probe> ObjectProbes;
StructuredBuffer<Probe> VolumeProbes;
StructuredBuffer<Beam> SunBeams;
Texture2D          CameraDepth;
Texture2D          VolumeTarget;
```

If object-probe interpolation is performed on the CPU, the object probe buffer need not remain resident on the GPU. The moving object receives only nine interpolated RGB SH coefficients.

B. Bake Kernels

The bake can retain a single HLSL compute source with separate build defines:

```
BUILD_LIGHTMAP_CS
BUILD_PROBE_CS
BUILD_SUN_VISIBILITY_CS
```

Random-number generation, ray-box intersection, ray-triangle intersection, closest-hit traversal, occlusion traversal, material evaluation, and path transport are shared.

This organization is more important than minimizing the number of shader source files. The objective is to prevent diverging transport implementations whose energy accounting or coordinate conventions gradually become inconsistent.

C. Probe Storage

During development, probe coefficients should remain FP32. After validation, serialization can convert them to FP16. Compression is intentionally delayed because basis, normalization, interpolation, and direction-convention errors are easier to diagnose without quantization.

D. Low-Resolution Volumetric Target

Direct and indirect volumetric lighting should be evaluated at reduced resolution. At 1280×720 output, a 320×180 target is a useful aggressive baseline. A depth-aware reconstruction pass rejects cross-surface interpolation and composites the reconstructed lighting before bloom and tone mapping.

The beam geometry already preserves the important high-frequency visibility boundaries, so low framebuffer resolution and high visibility resolution are not contradictory: they compress different dimensions of the problem.

XIII. COMPLEXITY AND MEMORY ANALYSIS

A. Static Surface Lighting

Runtime static-GI cost is one additional lightmap sample per shaded surface fragment:

$$O(N_{\text{visible fragments}}). \quad (28)$$

It is independent of bake sample count, path depth, and BVH complexity.

B. Moving-Object GI

When probe interpolation occurs once per object, eight neighboring probes and nine coefficients are combined:

$$O(8 \times 9) \quad (29)$$

coefficient operations per object update, followed by a small SH evaluation in the vertex or fragment shader.

C. Direct Volumetrics

Direct volumetric cost is approximately proportional to the low-resolution screen area covered by visible beam-cell proxies:

$$O(N_{\text{beam fragments}}). \quad (30)$$

There is no multiplicative factor corresponding to 32–128 fixed ray-march samples.

D. Indirect Volumetrics

For coarse grid traversal, cost is

$$O(N_{\text{lowres pixels}} \times N_{\text{crossed coarse cells}}). \quad (31)$$

The cell count is controlled by the deliberately sparse volumetric probe spacing.

E. Example Storage

For 4,000 object probes and 800 volume probes, FP16 SH coefficient storage is approximately

$$4000 \times 54 \approx 216 \text{ kB}, \quad (32)$$

$$800 \times 54 \approx 43 \text{ kB}. \quad (33)$$

Five thousand beam cells at 32 bytes each require approximately 160 kB. In a typical configuration, the lightmap remains the dominant baked-lighting allocation.

XIV. VALIDATION METHODOLOGY

A correct implementation should be validated representation by representation rather than only through the final image.

A. Spherical Harmonics

For a constant directional environment, only the $l = 0$ coefficient should remain non-zero after convergence. A synthetic directional lobe should reconstruct in the same orientation after projection and evaluation. These tests detect axis swaps, sign errors, and incorrect normalization.

The isotropic HG limit provides a second check: when $g = 0$, (18) becomes $1/(4\pi)$, and the SH phase convolution in (25) suppresses all bands above $l = 0$.

B. Volume Integration

Beer–Lambert transmittance should satisfy

$$T(a + b) = T(a)T(b) \quad (34)$$

and the segment integral should converge smoothly to its zero-extinction limit. A hard-coded homogeneous box under constant incident radiance provides a deterministic reference before sun beams are introduced.

C. Sun Visibility

A minimal scene containing one wall, one aperture, homogeneous fog, and one interior blocker is sufficient to validate the direct system. Beam cells must appear only behind the aperture and must terminate at the interior blocker. Thin bars crossing the aperture test whether the configured minimum sun feature size is preserved by the tiled visibility field and beam-cell compression.

D. Cross-Representation Energy

A diffuse moving object placed close to a static surface should receive probe irradiance consistent with the nearby static lightmap. Exact equality is not expected because the spatial representations differ, but large energy or color discrepancies usually indicate duplicated direct sun, missing $1/\pi$ factors, or SH normalization errors.

XV. EVALUATION PLAN

The implementation should ultimately be compared against a conventional volumetric baseline and a converged offline reference.

The primary measurements are:

- GPU frame time with volumetrics disabled and enabled;
- direct beam-cell fragment count;
- number and memory footprint of compressed beam cells;
- object and volume probe memory;
- low-resolution volume pass time;
- bake time for lightmaps, probes, and sun visibility;
- image error against an offline path-traced reference.

A useful test set should vary aperture size, blocker complexity, medium density, camera resolution, number of moving objects, and probe spacing. The system’s core hypothesis is not that volumetric lighting is free, but that expensive path and visibility complexity can be made largely a preprocessing concern, leaving a runtime cost that scales with compact reconstructed representations rather than the original transport problem.

XVI. LIMITATIONS

The design deliberately gives up several forms of dynamism.

First, a change to static geometry, sun direction, principal environment lighting, or baked medium geometry invalidates some or all precomputed data. Time-of-day systems therefore require multiple bakes, interpolation between bakes, or a different runtime method.

Second, low-order SH cannot reproduce narrow directional structures. The architecture addresses direct sunlight separately, but sharp indirect caustics or highly directional glossy interreflection remain outside the representation.

Third, a regular probe field can leak radiance through walls when spatial interpolation crosses disconnected regions. Visibility-weighted interpolation, probe relocation, region-aware grids, or tetrahedral connectivity can be introduced if this becomes visible.

Fourth, the analytic volumetric formulation assumes homogeneous or piecewise-homogeneous media. Fully dynamic smoke with rapidly changing density would require a separate density representation and likely reintroduce runtime sampling.

Fifth, sun visibility has a finite physical sampling resolution. The system can make that resolution extremely high during preprocessing, but it cannot mathematically guarantee preservation of geometry smaller than the configured minimum feature size.

Finally, the method does not eliminate runtime dynamic shadowing for moving geometry. It primarily targets the static environment and the indirect illumination received by dynamic objects.

XVII. DISCUSSION

The central design decision is not a particular basis, shader, or acceleration structure. It is the assignment of different lighting frequencies to different representations.

Lightmaps preserve high-frequency spatial variation on static surfaces. SH probes preserve smooth angular variation in world space. Beam cells preserve high-frequency direct visibility without requiring high-frequency directional basis functions. Analytic segment integration exploits the low complexity of homogeneous participating media. Each representation is therefore selected according to the dimensionality and frequency content of the transport it must retain.

This division also explains why a fully path-traced runtime is not necessarily the appropriate endpoint for

a renderer targeting weak systems. Path tracing remains valuable, but its value is shifted from final-image sampling to content compilation. More bake samples improve the stored solution without increasing player-machine frame cost.

The architecture is particularly suitable for constrained, mostly static environments in which lighting quality matters more than arbitrary lighting edits at runtime. Interior scenes with windows, corridors, rooms, strong color bleeding, and persistent atmospheric media are a natural target.

XVIII. CONCLUSION

This paper presented a fully precomputed light-transport architecture for combining path-traced static GI, dynamic-object indirect lighting, direct sun shafts, and indirect volumetric illumination on low-cost runtime hardware. A single bake-time BVH and path-tracing backend produces three compact classes of data: surface lightmaps, SH radiance probe fields, and compressed sun-beam cells. At runtime, static shading becomes a lightmap lookup, moving-object GI becomes low-order SH evaluation, and volumetric lighting becomes analytic integration over precomputed spatial domains.

The method deliberately avoids treating every lighting effect as a version of the same runtime problem. Smooth

transport is compressed; sharp visibility is retained explicitly; camera-independent work is moved offline; and homogeneous transport is integrated analytically. The resulting architecture is less dynamic than contemporary real-time GI systems, but its cost model is better aligned with hardware on which dense ray tracing, dense froxels, and high-sample volumetric ray marching are not practical.

REFERENCES

- [1] J. T. Kajiya, “The rendering equation,” in *Proc. SIGGRAPH*, 1986, pp. 143–150.
- [2] P.-P. Sloan, J. Kautz, and J. Snyder, “Precomputed radiance transfer for real-time rendering in dynamic, low-frequency lighting environments,” *ACM Trans. Graph.*, vol. 21, no. 3, pp. 527–536, 2002.
- [3] P.-P. Sloan, “Stupid spherical harmonics (SH) tricks,” Game Developers Conference, 2008.
- [4] T. Engelhardt and C. Dachsbacher, “Epipolar sampling for shadows and crepuscular rays in participating media with single scattering,” in *Proc. I3D*, 2010, pp. 119–125, doi: 10.1145/1730804.1730823.
- [5] S. Hillaire, “Physically-based and unified volumetric rendering in Frostbite,” in *SIGGRAPH Advances in Real-Time Rendering in Games Course*, 2015.
- [6] L. G. Henyey and J. L. Greenstein, “Diffuse radiation in the Galaxy,” *Astrophysical Journal*, vol. 93, pp. 70–83, 1941.
- [7] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4th ed. MIT Press, 2023.
- [8] Y. Qin, J. Lin, and X. Huang, “Massively parallel ray tracing algorithm using GPU,” arXiv:1504.03151, 2015.